LAPORAN PRAKTIKUM INTERNET OF THINGS

Praktikum 7

Oleh:

NAMA : Haikal Ahnaf Yahya

NPM : 25782098

KELAS : TRI – 3D

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026/2027

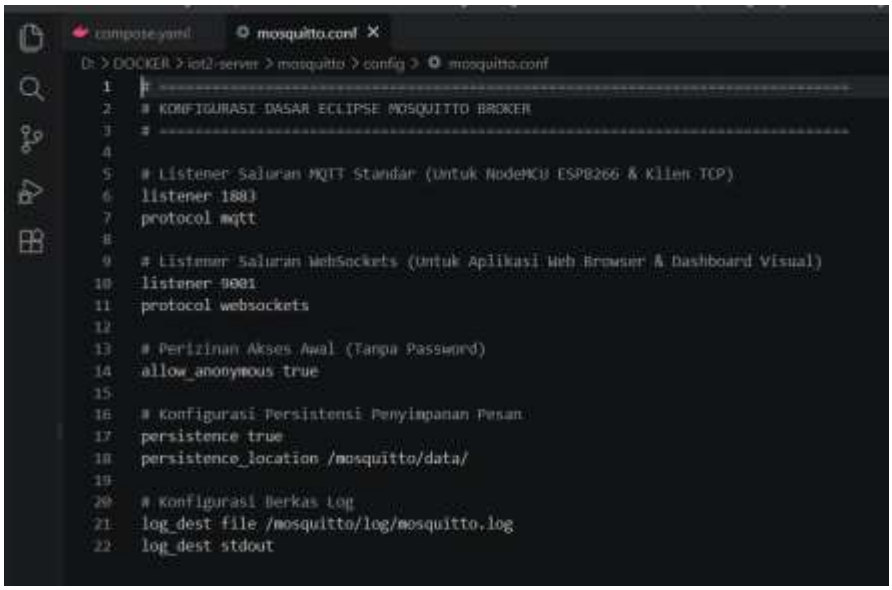
7.5 Praktikum

7.5.1 Praktikum 1: Pembangunan Ekosistem IoT Server dengan Docker Compose

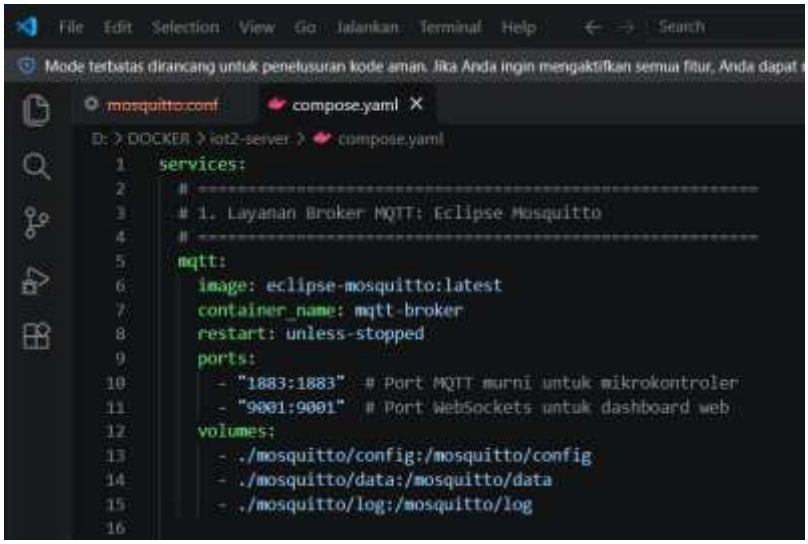
- buat direktori kerja baru

```
PS D:\DOCKER\iot2-server> tree
Folder PATH listing for volume New Volume
Volume serial number is B467-485E
D:.
├───mosquitto
│   ├───config
│   ├───data
│   └───log
└───nodered
    ├───lib
    │   └───flows
    └───node_modules
PS D:\DOCKER\iot2-server>
```

- buat berkas baru bernama mosquitto.conf



- buat berkas baru bernama docker-compose.yml

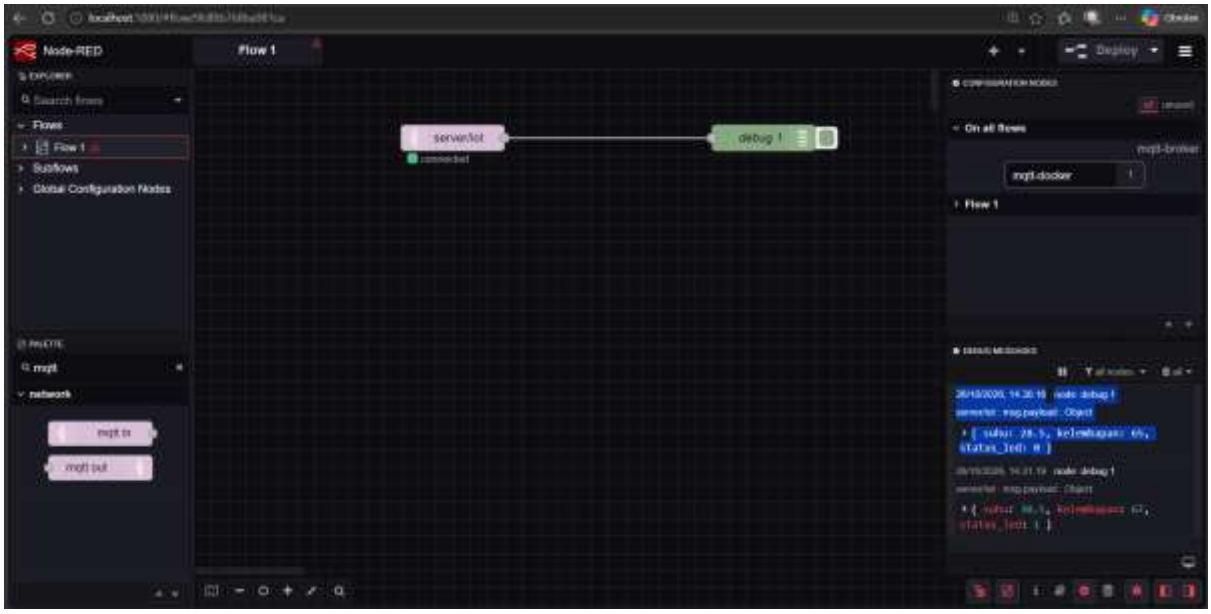


```
# ============================================================
# 2. Layanan Visual Rule Engine: Node-RED
# ============================================================
nodered:
  image: nodered/node-red:latest
  container_name: node-red
  restart: unless-stopped
  ports:
    - "1880:1880"  # Port antarmuka web builder Node-RED
  environment:
    - TZ=Asia/Jakarta
  volumes:
    - ./nodered:/data
  depends_on:
    - mqtt
```

- Eksekusi perintah berikut untuk mengunduh (pull) image dan menjalankan kontainer di latar belakang (detached mode)

```
PS D:\DOCKER\iot2-server> docker compose up -d
[+] up 0/2
 - Image eclipse-mosquitto:latest Pulling
[+] up 29/29
 ✓Image eclipse-mosquitto:latest Pulled
 ✓Image nodered/node-red:latest  Pulled
 ✓Network iot2-server_default    Created
 ✓Container mqtt-broker          Started
 ✓Container node-red             Started
PS D:\DOCKER\iot2-server> docker compose up -d
[+] up 2/2
 ✓Container node-red    Running
 ✓Container mqtt-broker Started
PS D:\DOCKER\iot2-server> docker compose up -d
[+] up 2/2
 ✓Container node-red    Running
 ✓Container mqtt-broker Started
PS D:\DOCKER\iot2-server>
```

- Buka peramban web pada laptop Anda dan ketikkan alamat URL: http://localhost:1880



7.5.2 Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal

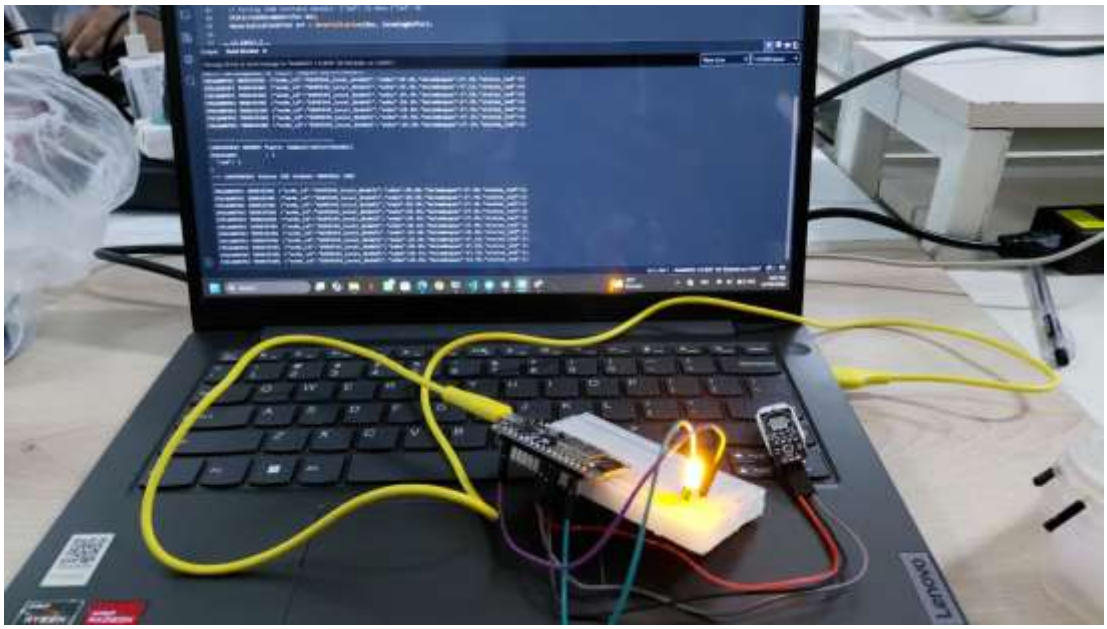
- Cari tahu Alamat IPv4 Lokal laptop Anda yang terhubung ke jaringan Wi-Fi:

```
Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::87af:8686:3177:b56b%20
   IPv4 Address. . . . . . . . . . . : 10.196.1.113
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 10.196.1.52

Ethernet adapter Bluetooth Network Connection:
```

- Hasil Rangkaian Perangkat Keras

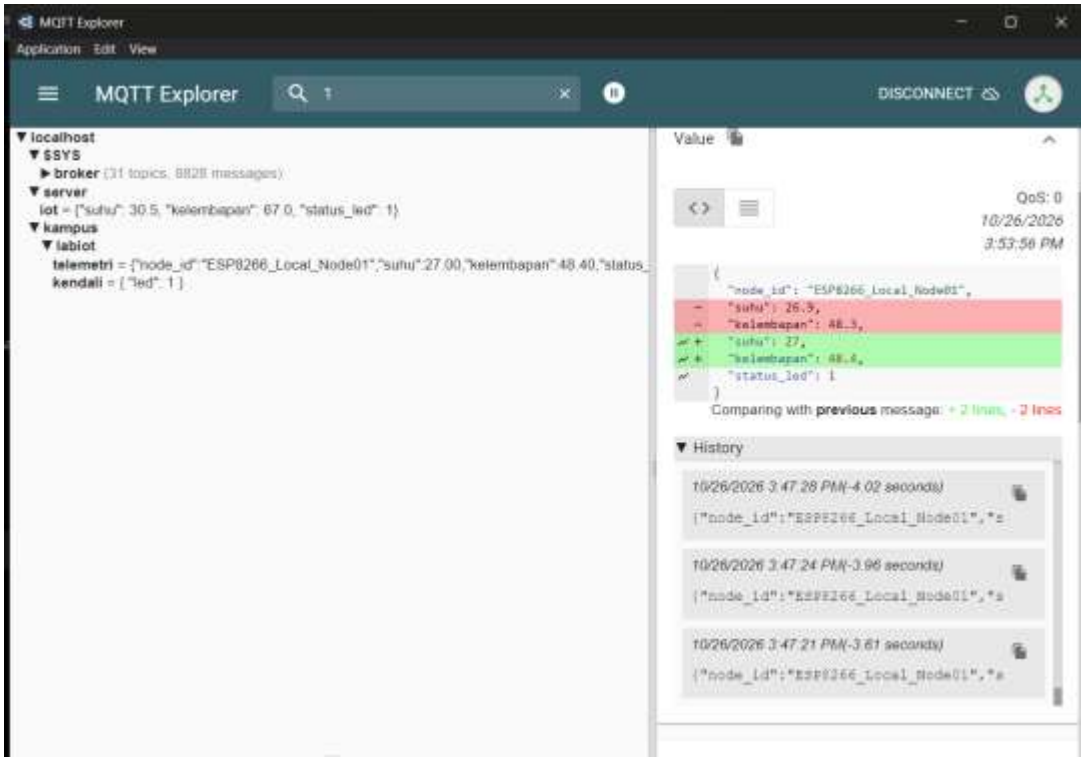


- Hasil Serial Monitor

```
.............
[Wi-Fi] Berhasil Terhubung!
[Wi-Fi] IP NodeMCU: 10.196.1.234
[MQTT] Menghubungkan ke Broker Lokal (10.196.1.113)... Terhubung!
[MQTT] Berlangganan ke topik: kampus/labiot/kendali
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.60,"kelembapan":47.30,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.60,"kelembapan":47.10,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.60,"kelembapan":47.00,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.60,"kelembapan":47.00,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.60,"kelembapan":47.00,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.60,"kelembapan":47.20,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.60,"kelembapan":47.20,"status_led":0}

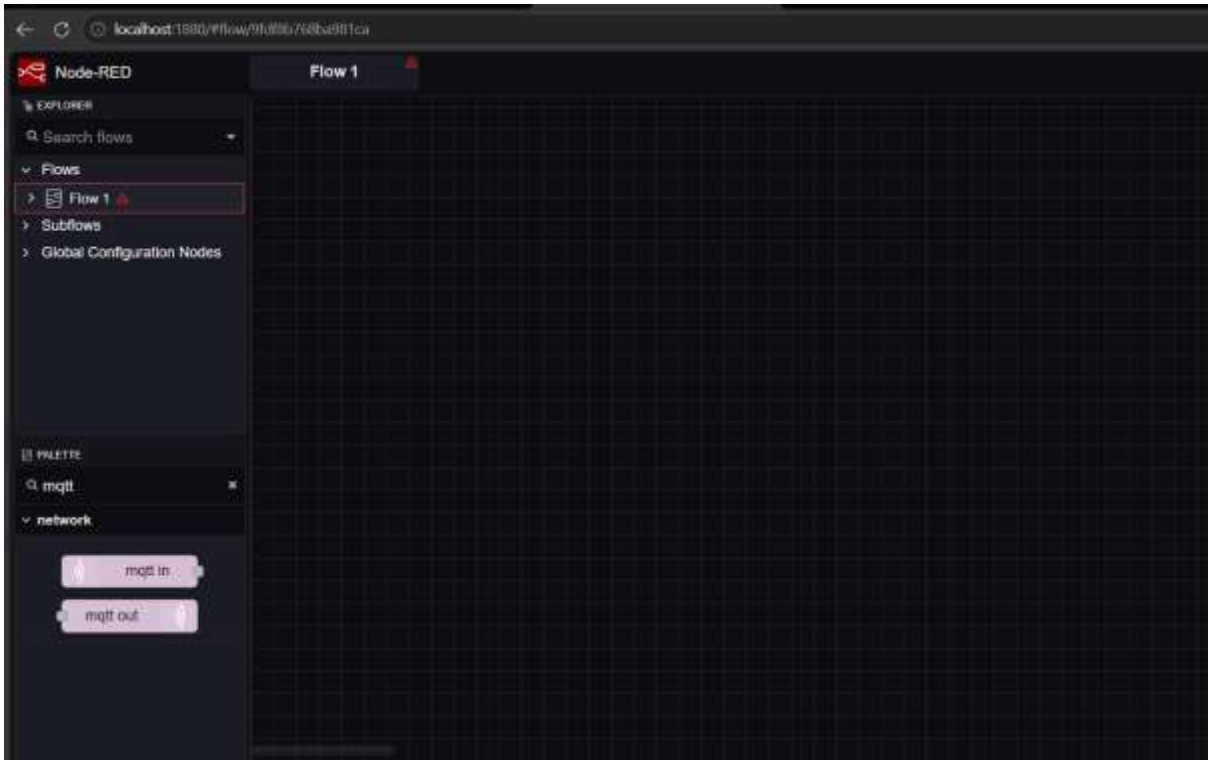
----------------------------------------
[INSTRUKSI MASUK] Topik: kampus/labiot/kendali
[PAYLOAD]         : {
  "led": 1
}
--> [AKTUATOR] Status LED diubah: MENYALA (ON)
----------------------------------------
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.60,"kelembapan":47.30,"status_led":1}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.60,"kelembapan":47.40,"status_led":1}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.60,"kelembapan":47.40,"status_led":1}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":28.60,"kelembapan":47.00,"status_led":1}
```

- Monitoring data menggunakan MQTT Explorer

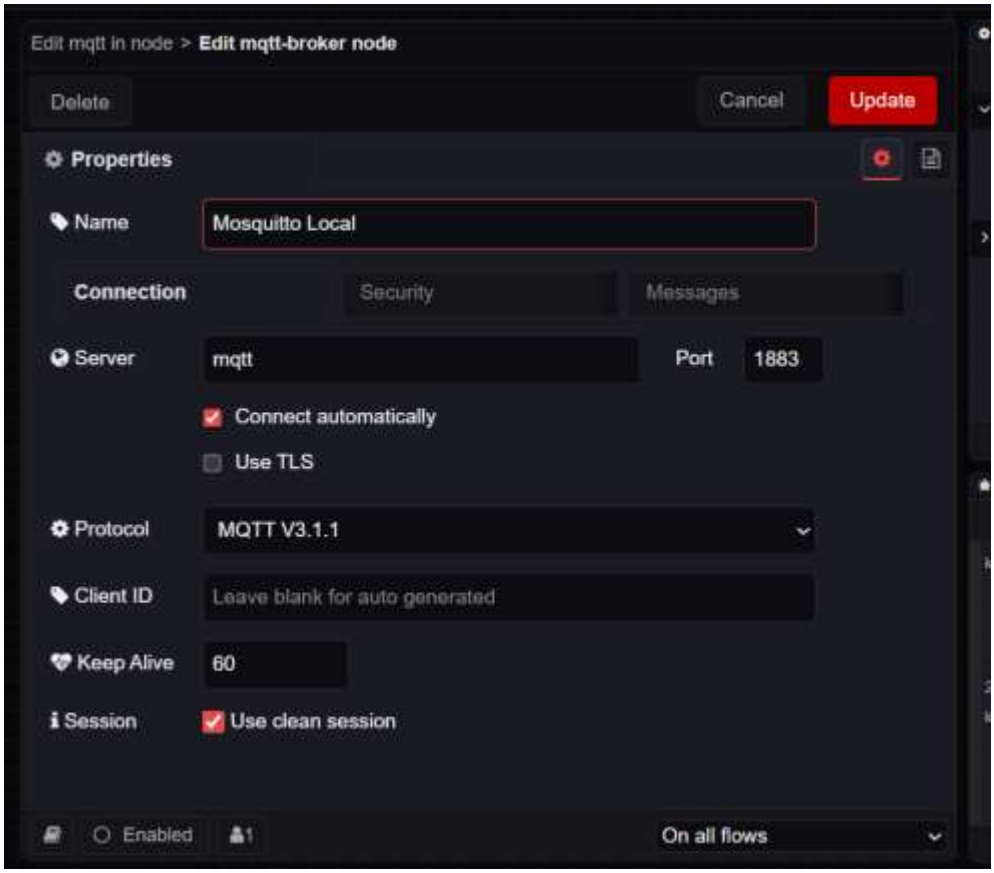


7.5.3 Praktikum 3: Integrasi Alur Data Telemetri Dua Arah pada Node-RED

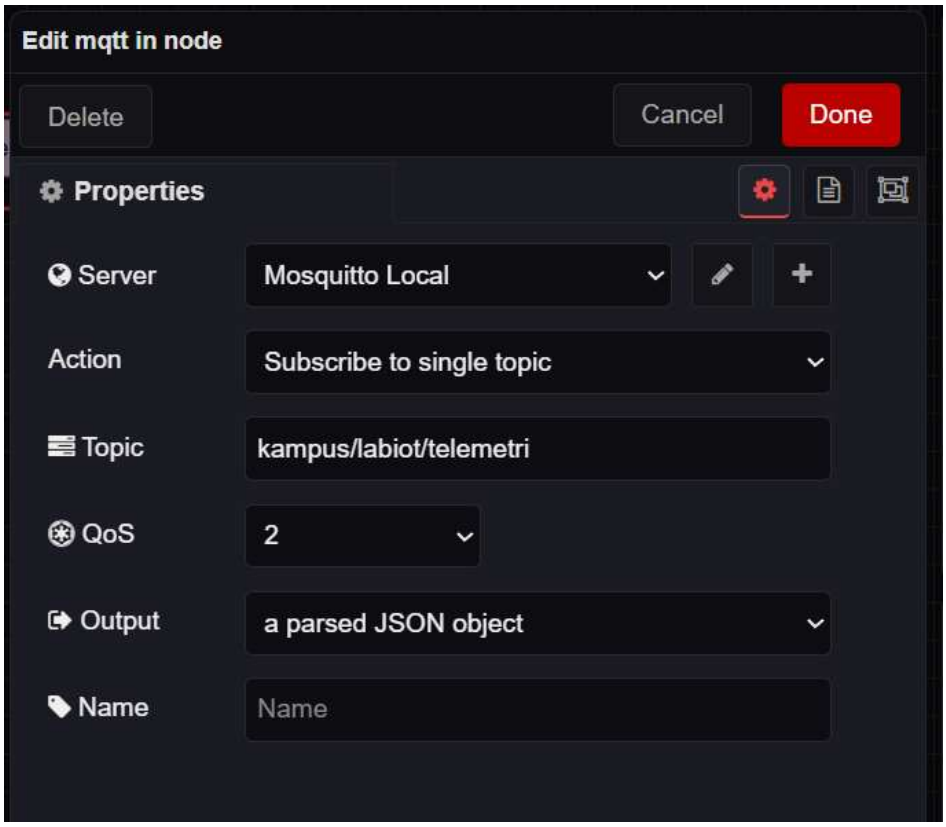
1. Buka peramban web dan akses antarmuka Node-RED di http://localhost:1880.



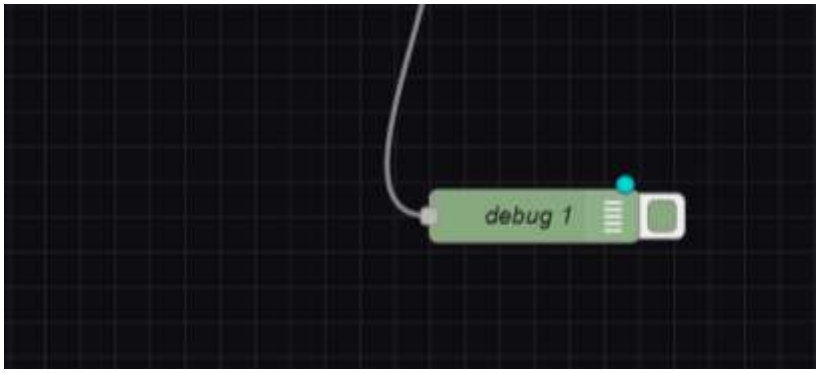
2. Menambahkan Konfigurasi Server MQTT Broker:



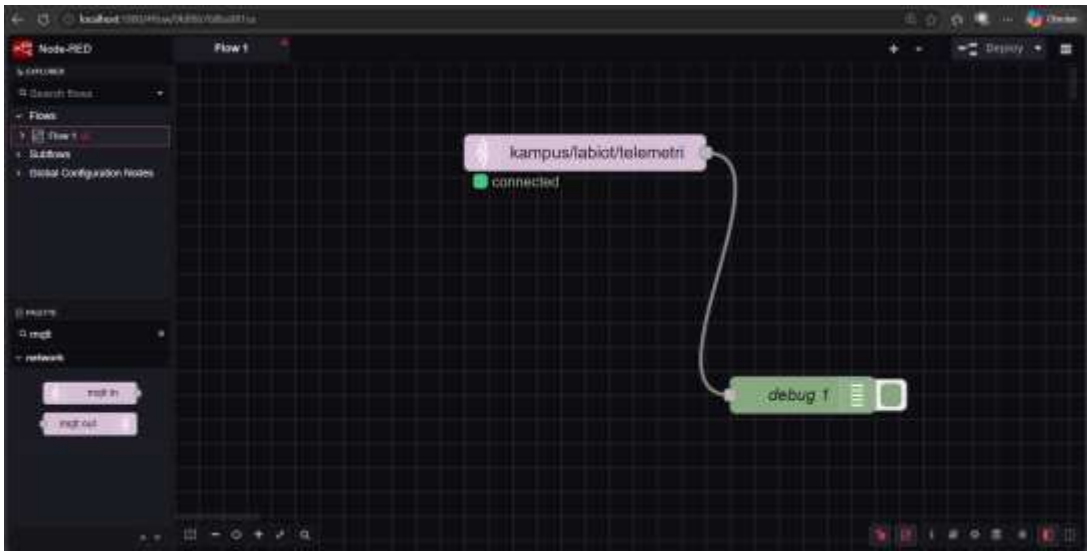
3. Mengonfigurasi Parameter Topik:



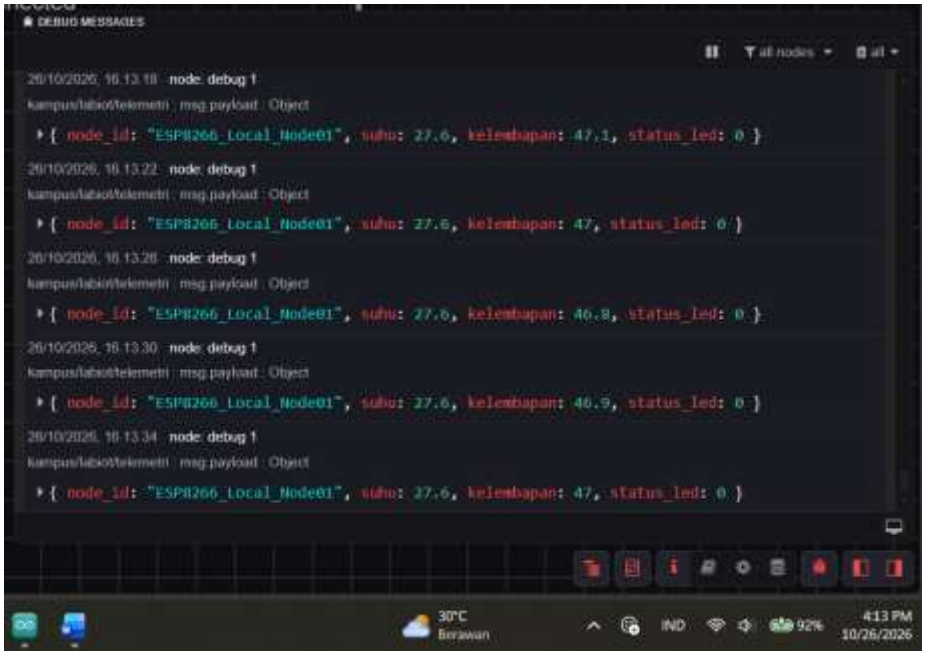
4. Menambahkan Node Pemantau (Debug Node):



5. Menerapkan Alur Kerja (Deploy)

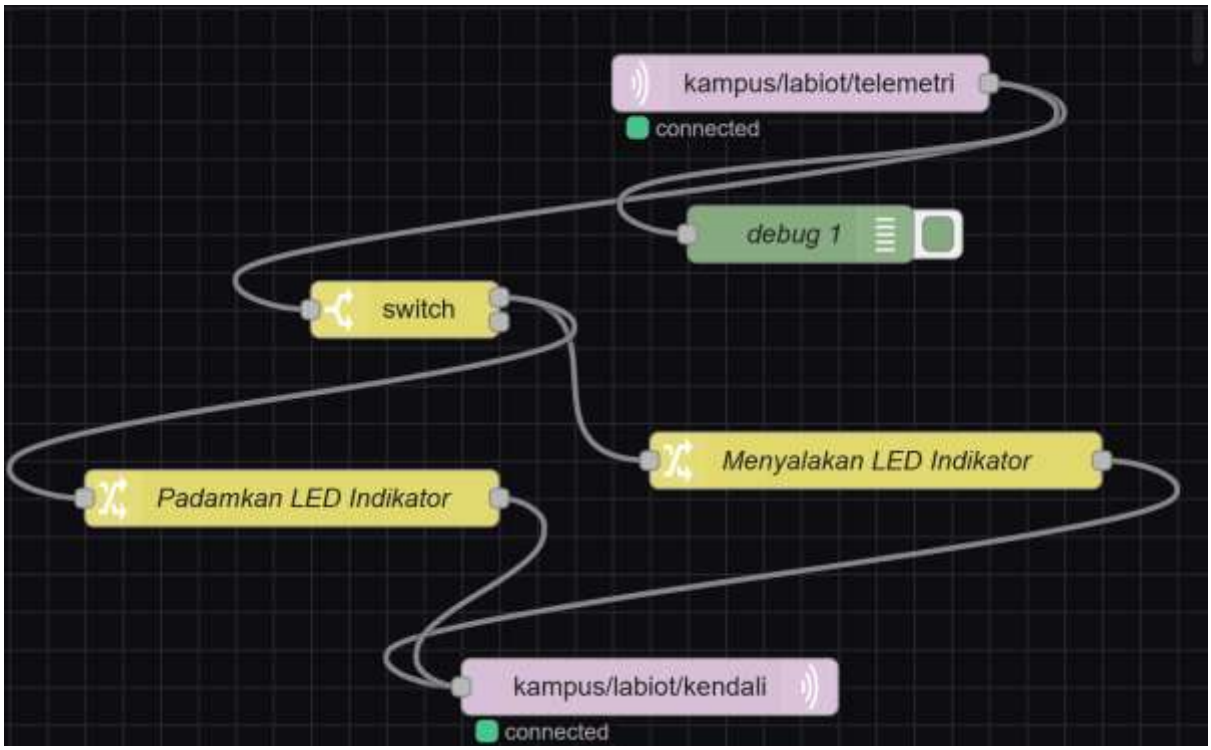


6. Menganalisis Aliran Data:

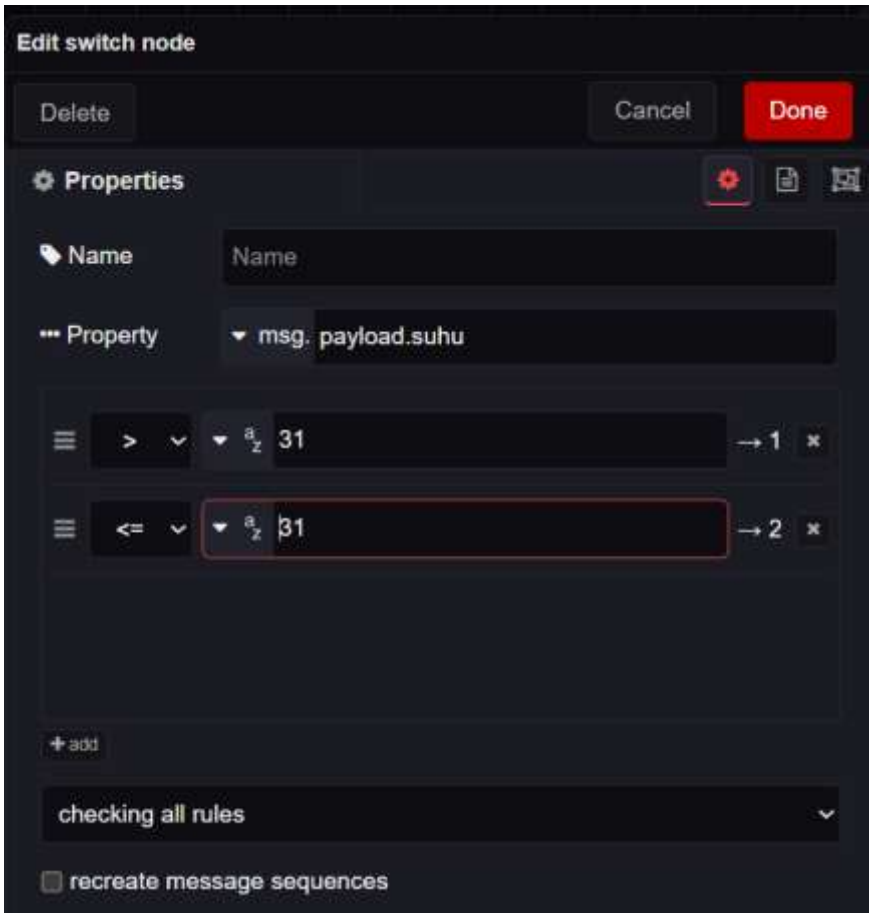


7.5.4 Praktikum 4: Implementasi Rule Engine Otomasi Aktuator Loop Tertutup

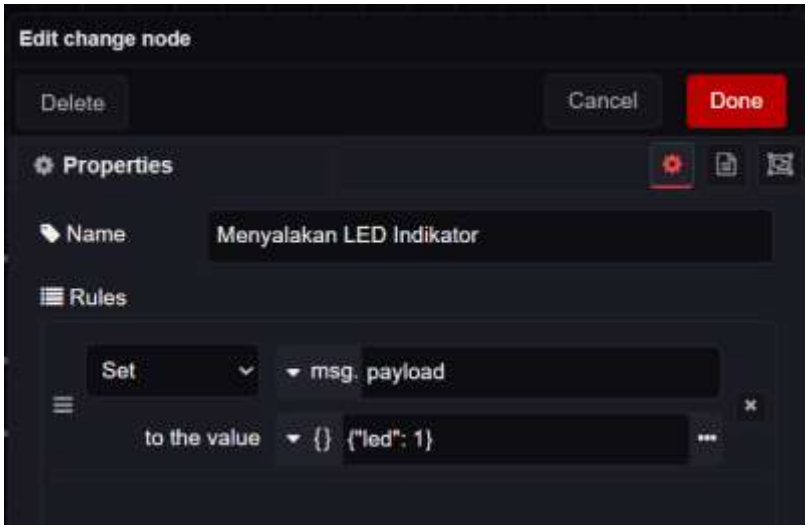
KANVAS ALUR LENGKAP: CLOSED-LOOP INDUSTRIAL RULE ENGINE



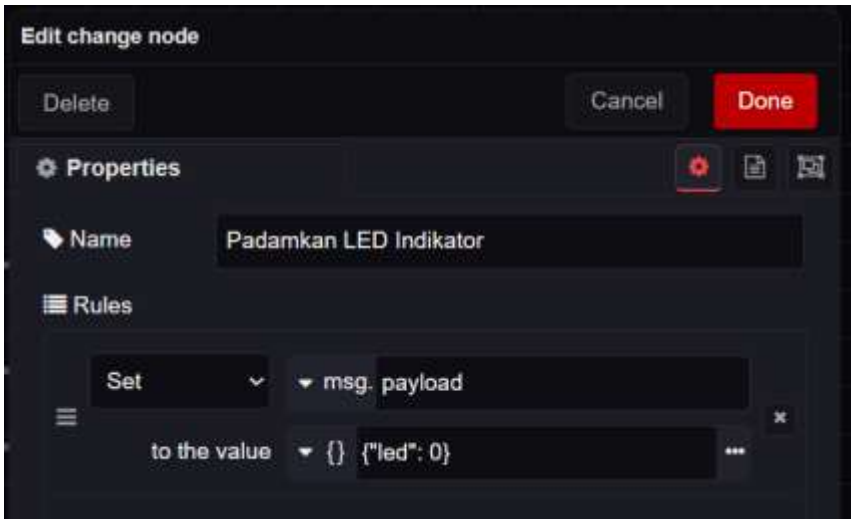
1. Membuat Node Percabangan Logika (switch)



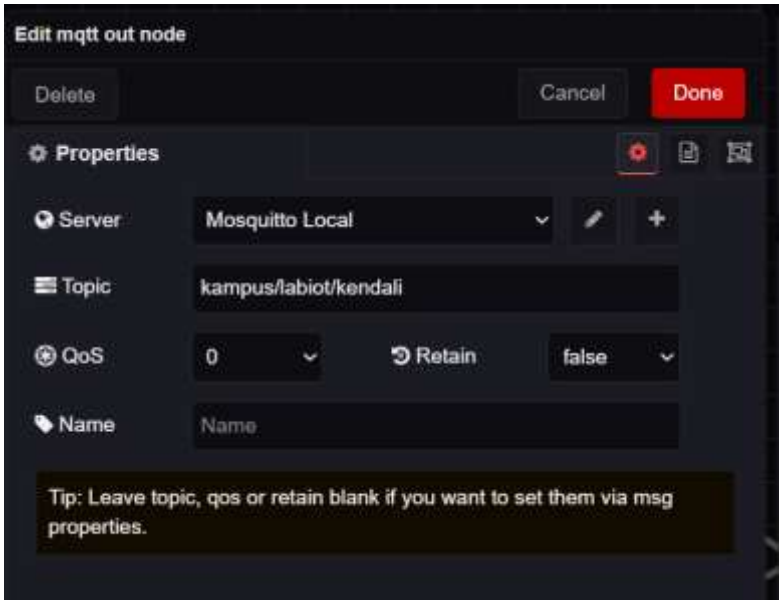
2. Membuat Node Transformasi Muatan Panas (change)



3. Membuat Node Transformasi Muatan Normal (change)



4. Menerbitkan Instruksi Balik ke Broker (mqtt out)



5. Pengujian Fisik Sistem Otomasi Tertutup

Saat suhu dibawah 31, Node-RED mengirim {"led":0} dan LED padam.

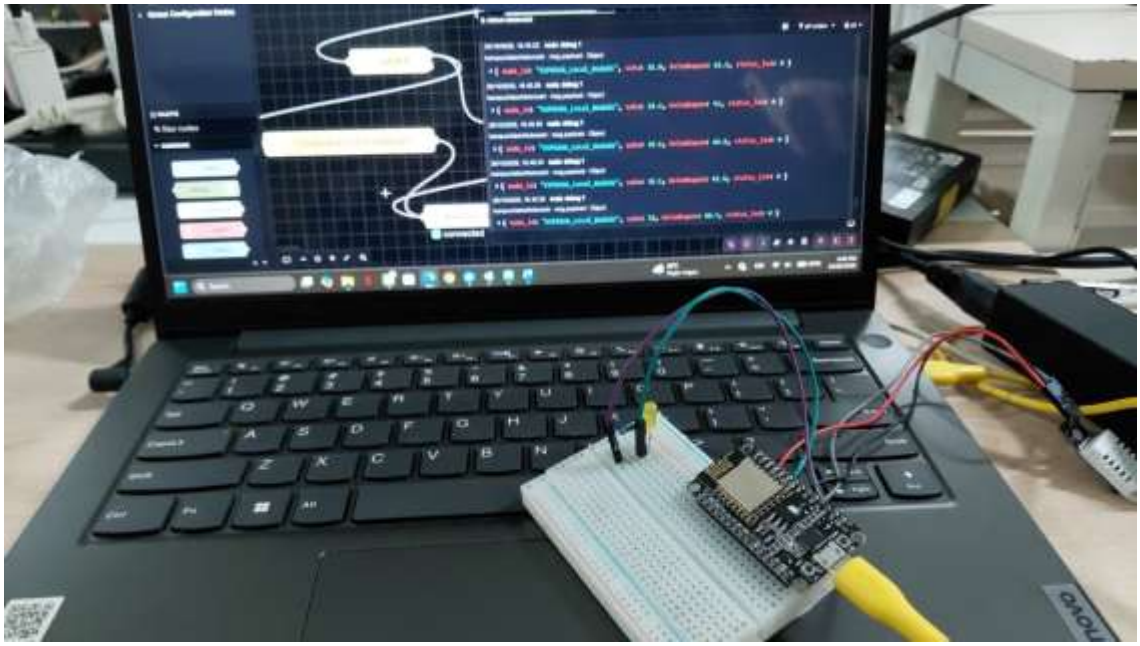
- Serial monitor

```
----------------------------------------
[INSTRUKSI MASUK] Topik: kampus/labiot/kendali
[PAYLOAD]         : {"led":0}
--> [AKTUATOR] Status LED diubah: PADAM (OFF)
----------------------------------------
```

- Hasil Nodered debug



- Hasil Rangkain Fisik



Saat suhu dibawah 31, Node-RED mengirim {"led":1} dan LED nyala.

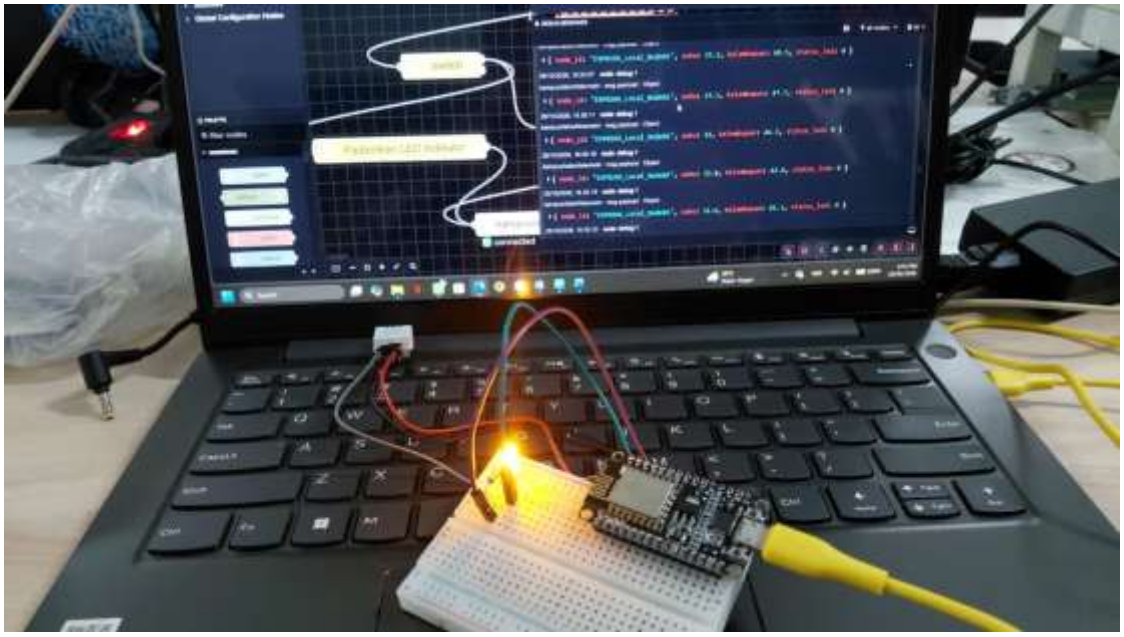
- Serial monitor

```
> ----------------------------------------
> [INSTRUKSI MASUK] Topik: kampus/labiot/kendali
> [PAYLOAD]         : {"led":1}
> --> [AKTUATOR] Status LED diubah: MENYALA (ON)
> ----------------------------------------
>
```

- Hasil Nodered debug



- Hasil Rangkain Fisik



7.6 Latihan

Latihan 1 — Analisis Jaringan Internal Docker dan Service Discovery

1. Mengapa localhost menyebabkan kegagalan koneksi di dalam kontainer Node-RED?

- localhost mengacu pada perangkat atau lingkungan jaringan tempat program berjalan. Karena Node-RED berada di dalam kontainer Docker, localhost menunjuk ke kontainer Node-RED itu sendiri, bukan ke kontainer Mosquitto. Akibatnya, Node-RED tidak menemukan broker MQTT pada alamat tersebut dan koneksi dapat gagal dengan pesan Connection Refused.

2. Bagaimana Docker Embedded DNS Server dan Bridge Network bekerja?

- Docker Compose menghubungkan kontainer yang berada dalam jaringan yang sama melalui jaringan Docker, umumnya menggunakan driver bridge. Docker Embedded DNS Server menerjemahkan nama servis, seperti mqtt, menjadi alamat IP internal kontainer broker. Dengan demikian, Node-RED dapat mengakses Mosquitto melalui nama servis mqtt pada port 1883 tanpa perlu mengetahui alamat IP internal kontainer secara langsung.

Latihan 2 — Penanganan Masalah Konektivitas Jaringan dan Subnet Firewall

1. Mengapa NodeMCU gagal terhubung ke broker MQTT?

- Alamat IP NodeMCU adalah 192.168.43.102, sedangkan IP laptop adalah 192.168.1.15. Jika keduanya menggunakan subnet mask 255.255.255.0, NodeMCU berada pada subnet 192.168.43.0/24, sedangkan laptop berada pada subnet 192.168.1.0/24. Keduanya berada pada subnet yang berbeda sehingga tidak dapat berkomunikasi secara langsung tanpa routing yang sesuai.

2. Apa langkah teknis untuk mengatasinya?

- Mahasiswa perlu menghubungkan laptop dan NodeMCU ke jaringan Wi-Fi atau hotspot yang sama, kemudian memastikan keduanya memperoleh alamat IP dalam subnet yang sama. Setelah itu, alamat IP laptop yang benar dimasukkan ke konfigurasi mqttServer pada program ESP8266. Pastikan port TCP 1883 terbuka dan koneksi tidak diblokir firewall maupun fitur isolasi klien Wi-Fi.

Latihan 3 (Implementasi Filter Histeresis pada Rule Engine):

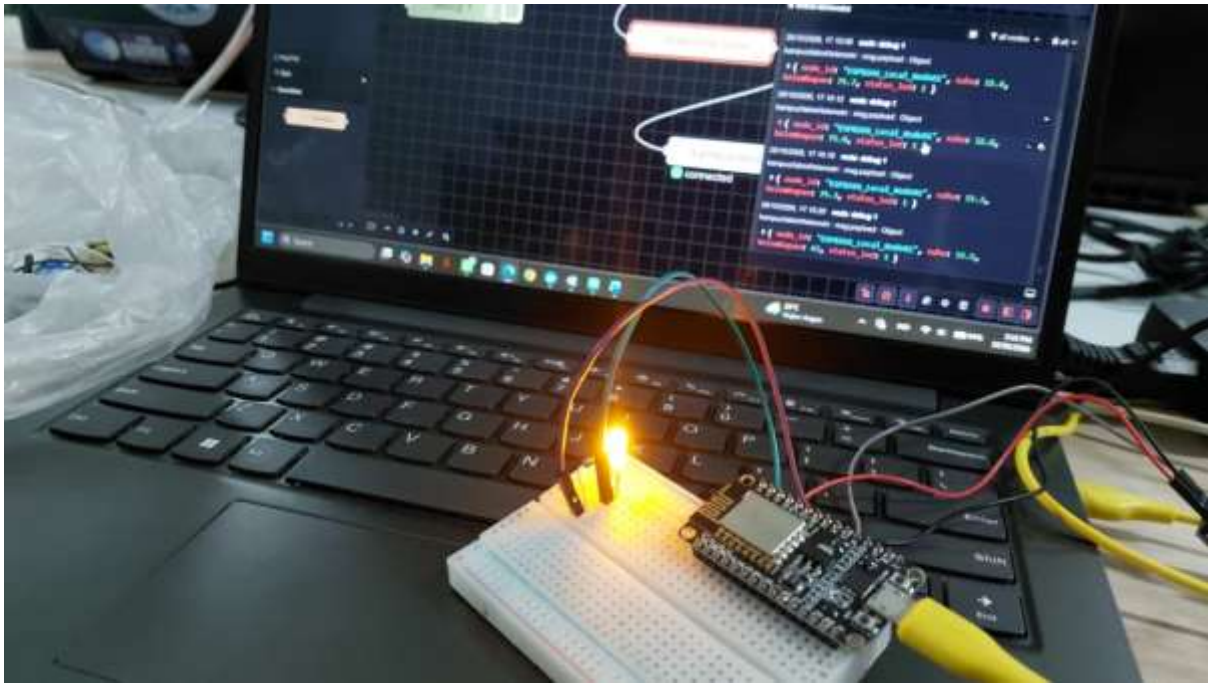
```
const suhu = Number(msg.payload.suhu);
if (!Number.isFinite(suhu)) {
    return null;
}

let led = context.get("ledState");
if (led === undefined) {
    led = 0;
}
if (suhu > 31.5) {
    led = 1;
} else if (suhu < 30.0) {
    led = 0;
} else {
    return null;
}
```

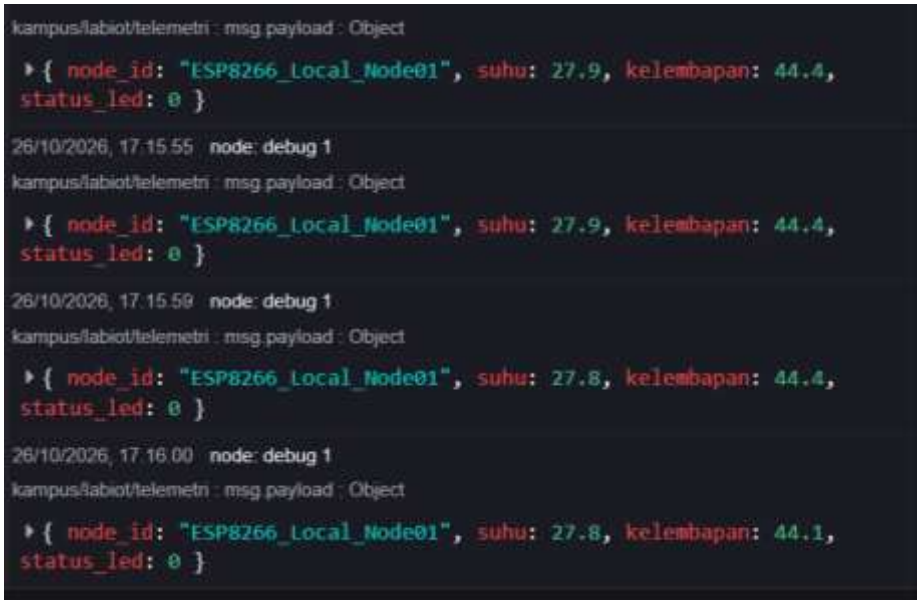
```
17
18    context.set("ledState", led);
19
20    msg.payload = {
21        led: led
22    };
23
24    return msg;
25
```

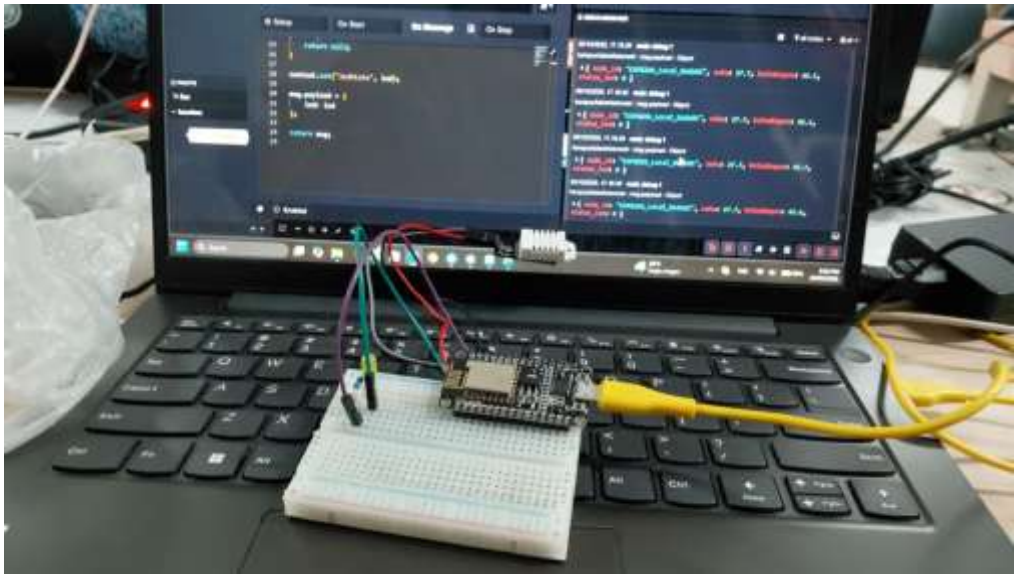
- Suhu lebih dari 31.5°C → LED menyala.

```
{ node_id: "ESP8266_Local_Node01", suhu: 32.1, kelembapan: 75.7,
status_led: 1 }
26/10/2026, 17.10.08  node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 32.4, kelembapan: 75.7,
status_led: 1 }
26/10/2026, 17.10.12  node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 32.8, kelembapan: 75.9,
status_led: 1 }
26/10/2026, 17.10.16  node: debug 1
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 33.3, kelembapan: 75.3,
status_led: 1 }
```



- Suhu kurang dari 30.0°C → LED padam





Latihan 4 — Analisis Volume Mount dan Persistensi Data

1. Apa yang terjadi jika baris volume dihapus dari docker-compose.yml?

- Jika volume ./nodered:/data dihapus, data Node-RED seperti flow, konfigurasi, dan kredensial yang disimpan di /data hanya berada pada lapisan penyimpanan kontainer. Ketika kontainer dihapus menggunakan docker compose down dan dibuat kembali, data tersebut dapat hilang. Akibatnya, flow yang telah dibuat mungkin tidak tersedia lagi.

2. Apa perbedaan Bind Mount dan Named Volume?

Bind Mount adalah metode penyimpanan Docker yang menghubungkan direktori atau file tertentu di komputer host dengan direktori di dalam kontainer. Contohnya adalah ./nodered:/data. Metode ini memudahkan pengguna mengakses dan mengelola file secara langsung melalui folder di komputer.

Sementara itu, Named Volume adalah metode penyimpanan yang dikelola oleh Docker. Contohnya adalah nodered_data:/data. Penyimpanan ini cocok untuk menyimpan data aplikasi secara persisten tanpa perlu menentukan direktori penyimpanan host secara langsung.

Keduanya dapat mempertahankan data meskipun kontainer dihapus, selama penyimpanan tersebut tetap digunakan dan datanya tidak ikut dihapus. Named Volume biasanya tetap tersimpan setelah perintah docker compose down, kecuali dihapus secara eksplisit menggunakan perintah docker compose down -v.

7.7 Tugas: Pengamanan Server IoT Industri (MQTT Authentication & Access Control)

1. Membuat Berkas Kredensial Terenkripsi di Dalam Kontainer:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS D:\DOCKER\iot2-server\mosquitto\config> docker exec -it mqtt-broker mosquitto_passwd -c /mosquitto/config/password.tx
t admin
Password:
Reenter password:
Error: Passwords do not match.

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug mqtt-broker
    Learn more at https://docs.docker.com/go/debug-cli/
PS D:\DOCKER\iot2-server\mosquitto\config> docker exec -it mqtt-broker mosquitto_passwd -c /mosquitto/config/password.tx
t admin
Password:
Reenter password:
Adding password for user admin

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug mqtt-broker
    Learn more at https://docs.docker.com/go/debug-cli/
PS D:\DOCKER\iot2-server\mosquitto\config>
```

2. Memperbarui Konfigurasi Kebijakan Mosquitto:

```
# KONFIGURASI DASAR ECLIPSE MOSQUITTO BROKER

# Listener MQTT untuk NodeMCU ESP8266 dan klien TCP
listener 1883
protocol mqtt

# Listener WebSockets untuk dashboard
listener 9001
protocol websockets

# Pengamanan dengan username dan password
allow_anonymous false
password_file /mosquitto/config/password.txt

# Persistensi penyimpanan pesan
persistence true
persistence_location /mosquitto/data/

# Konfigurasi log
log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
```

3. Memuat Ulang Layanan Broker:

```
PS D:\DOCKER\iot2-server\mosquitto\config> docker restart mqtt-broker
mqtt-broker
```

4. Rekayasa Perbaikan Sistem (Penyelesaian Tugas):

Pemutakhiran Skrip Program NodeMCU ESP8266

```
// Menghubungkan ke MQTT dengan autentikasi
void reconnectMQTT() {
  while (!client.connected()) {
    Serial.printf("[MQTT] Menghubungkan ke %s...\n", mqttServer);

    if (client.connect(clientId, mqttUser, mqttPassword)) {
      Serial.println("[MQTT] Berhasil terhubung dengan autentikasi!");

      if (client.subscribe(topicSubCmd)) {
        Serial.printf("[MQTT] Berlangganan: %s\n", topicSubCmd);
      } else {
        Serial.println("[MQTT] Gagal berlangganan topik kendali.");
      }
    } else {
      Serial.printf("[MQTT] Gagal, rc=%d. Coba lagi 4 detik.\n",
                    client.state());
      delay(4000);
    }
```

- Hasil Serial monitor

```
[INSTRUKSI MASUK] Topik: kampus/labiot/kendali
[PAYLOAD]: {"led":0}
[AKTUATOR] LED PADAM
----------------------------------------
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":27,"kelembapan":46.1,"status_led":0}

----------------------------------------
[INSTRUKSI MASUK] Topik: kampus/labiot/kendali
[PAYLOAD]: {"led":0}
[AKTUATOR] LED PADAM
----------------------------------------
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":27.1,"kelembapan":46.2,"status_led":0}

----------------------------------------
[INSTRUKSI MASUK] Topik: kampus/labiot/kendali
[PAYLOAD]: {"led":0}
[AKTUATOR] LED PADAM
----------------------------------------
```

Pemutakhiran Konfigurasi Kredensial pada Node-RED

